OS202 – Systemes paralleles et distribues

Optimisation Par Colonie De Fourmi D'un Chemin Sur Un Paysage Fractal

MENESES GAMBOA Carlos

VASQUEZ TORRES Jair

Ecole Nationale des techniques avancees

Mars 2026

Table des matières

1 Introduction et environnement d'execution **3**
1.1 Methodologie experimentale et reproductibilite . . . 3
1.2 Machine de test . . . 3
1.3 Organisation du depot . . . 4

2 Q0 : Analyse de l'existant et baseline monocoeur **4**
2.1 Pipeline applicatif . . . 4
2.2 Hotspots theoriques . . . 4
2.2.1 Avance des fourmis . . . 4
2.2.2 Traitement global de la grille . . . 5
2.3 Baseline sequentielle . . . 5

3 Q1 : Instrumentation et definition du scope **5**
3.1 Instrumentation par `std::chrono` . . . 5
3.2 Sanity checks et scope de mesure . . . 6
3.3 Tableau de resultats Q1 . . . 6

4 Q2 : Vectorisation (AoS vs SoA) et optimisations **6**
4.1 Theorie : passage de AoS a SoA . . . 6
4.2 Le paradoxe des performances : pourquoi SoA n'a pas apporte un gain magique . 7
4.3 Les vraies optimisations du code final . . . 7
4.3.1 Preallocation des structures auxiliaires . . . 7
4.3.2 Separation des fourmis selon l'etat . . . 8
4.3.3 Reduction du cout du generateur pseudo-aleatoire . . . 8
4.3.4 Autres ameliorations structurelles . . . 8
4.4 Tableau comparatif Q1 vs Q2 dans le scope compute . . . 8
4.5 Reproductibilite et validation fonctionnelle . . . 9
4.6 Conclusion Q2 . . . 9

5 Q3 : Parallelisation en Memoire Partagee (OpenMP) **9**
5.1 Strategie generale et protocole experimental . . . 9
5.2 Surmonter le goulet d'etranglement des verrous . . . 10
5.3 Ordonnancement OpenMP retenu . . . 11
5.4 Resultats et analyse quantitative . . . 11
5.5 Analyse des figures de scalabilite . . . 12
5.5.1 Regime 1 thread : overhead pur . . . 13

5.5.2 Regime utile : 2 a 4 threads 13
5.5.3 Regime 8 threads : plafond materiel et Hyperthreading 14
5.6 Conclusion Q3 14

1 Introduction et environnement d'execution

Ce projet etudie l'optimisation d'une simulation de colonie de fourmis sur paysage fractal, avec une progression volontairement structuree en quatre etapes. Q0 etablit la baseline monocoeur et l'analyse des hotspots, Q1 introduit l'instrumentation temporelle, Q2 restructure les donnees et simplifie le noyau chaud, puis Q3 ajoute une parallelisation OpenMP en memoire partagee. Le fil directeur du rapport est de distinguer clairement les gains dus a la mesure, a l'organisation memoire, puis a la parallelisation, tout en verifiant a chaque etape la reproductibilite fonctionnelle.

1.1 Methodologie experimentale et reproductibilite

> Le protocole vise une comparaison reproductible. La graine initiale est fixe a `2026` dans les executables Qx. Un *warm-up* complet est execute avant les repetitions mesurees afin de chauffer les caches et n'est jamais comptabilise dans les resultats exportes. La validation fonctionnelle repose sur deux indicateurs : `Food_KPI`, qui compte la nourriture ramenee au nid, et un `Checksum` construit a partir de l'etat final des fourmis et des pheromones. Toute optimisation retenue doit conserver ces deux quantites.

Le protocole scientifique vise un **scope compute strict**, c'est-a-dire des mesures concentrees sur la boucle de calcul avec rendu SDL desactive via `-gui 0`. En pratique, le depot conserve deux familles de campagnes : les campagnes historiques Q0 et Q1, qui correspondent a des executions completes avec influence du rendu, et les campagnes finales Q2 et Q3, qui ont ete consolidees en mode headless pour isoler le calcul. Le rapport signale explicitement cette difference afin d'eviter toute comparaison abusive entre temps applicatifs complets et temps purement compute.

Toutes les campagnes exploitees dans ce document reposent sur **5 repetitions** avec warm-up prealable, mais leur perimetre de mesure n'est pas uniforme entre les premieres etapes et les campagnes finales.

1.2 Machine de test

Le fichier `results/machine_info.txt` n'est pas present dans l'archive courante. Les informations ci-dessous ont donc ete reconstituees a partir de l'environnement d'execution local et des metadonnees deja presentes dans le depot.

Table 1 – Environnement d'execution utilise pour les mesures et l'analyse.

Champ	Valeur
Systeme	Linux 6.6.87.2-microsoft-standard-WSL2 x86_64
Processeur	11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
Coeurs physiques	4
Coeurs logiques	8
Threads par coeur	2
Cache L1d / L1i	192 KiB / 128 KiB
Cache L2	5 MiB
Cache L3	8 MiB
Memoire vive	7.6 GiB
Compilateur C++	g++ 11.4.0

1.3 Organisation du depot

Le depot est structure par etapes independantes :

- `Q0/` : version de reference monocoeur ;
- `Q1/` : version instrumentee avec decomposition temporelle ;
- `Q2/` : version SoA et optimisations du noyau chaud ;
- `Q3/` : version OpenMP en memoire partagee ;
- `results/` : mesures historiques et campagnes compute intermediaires ;
- `final_results/` : campagne finale Q3 et figures de scalabilite.

Cette organisation garantit la tracabilite des modifications et facilite la comparaison entre versions.

2 Q0 : Analyse de l'existant et baseline monocoeur

2.1 Pipeline applicatif

Le pipeline global de l'application peut etre resume comme suit :

Generation du terrain fractal
$\Downarrow$
Initialisation des pheromones et des fourmis
$\Downarrow$
Boucle de simulation
deplacement des fourmis $\rightarrow$ evaporation $\rightarrow$ mise a jour des pheromones $\rightarrow$ rendu / sorties
$\Downarrow$
Export des temps, du `Food_KPI` et du `Checksum`

Cette decomposition met deja en evidence trois blocs de calcul dominants :

- le deplacement des fourmis, avec une complexite proportionnelle au nombre de fourmis et a leur budget de mouvement ;
- l'evaporation globale, qui parcourt la grille complete ;
- la mise a jour des pheromones, elle aussi tres dependante de la taille de la grille.

2.2 Hotspots theoriques

Avant toute instrumentation, deux goulots d'etranglement etaient deja identifies.

2.2.1 Avance des fourmis

Le deplacement des fourmis constitue le principal candidat au statut de point chaud theorique. Chaque iteration parcourt toutes les fourmis, lit les pheromones voisines, applique une decision exploration / exploitation, met a jour l'etat de charge et peut effectuer plusieurs micro-deplacements avant d'epuiser le budget de temps. Cette phase est a la fois :

— intensive en branchements ;
— sensible a la localite memoire ;
— dependante du generateur pseudo-aleatoire.

2.2.2 Traitement global de la grille

L'evaporation et la mise a jour des pheromones parcourent la grille entiere a chaque iteration. Leur cout est donc structurellement lie a la taille du paysage fractal. Ce sont des noyaux plus reguliers que le deplacement des fourmis, mais potentiellement couteux en trafic memoire.

2.3 Baseline sequentielle

Le fichier `results/Q0_baseline.csv` contient la baseline monocoeur archivee. Cette mesure correspond a une execution applicative complete de reference, anterieure a l'isolation stricte du scope compute.

Table 2 – Baseline sequentielle Q0 extraite de `results/Q0_baseline.csv`.

Metrique	Valeur
Nombre de repetitions archivees	5
Temps moyen T_s	85 520.09 ms
Ecart-type	2384.22 ms
Premiere iteration avec nourriture	1236
Food_KPI	4353
Checksum	1479050492479337403

Cette baseline sert de point de reference algorithmique et applicatif. Elle montre que la simulation complete se situe deja dans un regime de calcul non trivial, tout en rappelant qu'a ce stade le rendu reste melange au temps total observe.

3 Q1 : Instrumentation et definition du scope

3.1 Instrumentation par `std::chrono`

L'etape Q1 introduit une instrumentation par sous-etapes a l'aide de `std::chrono`. Le temps total `T_total` encadre la boucle temporelle principale, tandis que quatre sous-mesures sont isolees :

— `T_move_ants` : deplacement des fourmis ;
— `T_evap` : evaporation globale des pheromones ;
— `T_pher_update` : application de la nouvelle carte de pheromones ;
— `T_render` : rendu SDL.

Ce decoupage permet de distinguer :

— les noyaux calculatoires internes ;
— le cout de l'interface graphique ;
— la coherence globale entre la somme des sous-etapes et le temps total.

3.2 Sanity checks et scope de mesure

Le code affiche un *sanity check* pour verifier que :

$$T_{\text{total}} \approx T_{\text{move_ants}} + T_{\text{evap}} + T_{\text{pher_update}} + T_{\text{render}}.$$

En mode headless, un second controle impose $T_{\text{render}} \approx 0$. Ce point correspond au scope experimental vise par le projet : isoler strictement le calcul.

Cependant, les CSV archives dans `results/` pour Q1 contiennent un `T_render` non nul. Les valeurs reportees ci-dessous reproduisent donc fidelement la campagne disponible, et non une campagne recalculée en `-gui 0`.

3.3 Tableau de resultats Q1

TABLE 3 – Decomposition temporelle Q1 extraite de `results/Q1_timings_breakdown.csv`.

Sous-etape	Moyenne (ms)	Ecart-type (ms)	Part dans T_{total}
$T_{\text{move_ants}}$	15253.78	1898.42	18.13 %
T_{evap}	1664.29	48.18	1.98 %
$T_{\text{pher_update}}$	38.92	3.79	0.05 %
T_{render}	67094.22	1198.94	79.76 %
T_{total}	84118.09	2178.69	100.00 %

Les indicateurs fonctionnels associes a cette campagne sont :

- `First_Iteration = 1236`
- `Food_KPI = 4353`
- `Checksum = 1479050492479337403`

La conclusion operationnelle de Q1 doit etre lue a deux niveaux. Sur le temps applicatif complet archive, le rendu domine largement le wall-time. En revanche, a l'interieur du scope compute vise par le projet, le noyau `move_ants` reste de loin le principal hotspot calculatoire, devant l'evaporation et la mise a jour des pheromones.

4 Q2 : Vectorisation (AoS vs SoA) et optimisations

4.1 Theorie : passage de AoS a SoA

L'objectif theorique de Q2 etait de transformer le stockage des fourmis depuis une organisation de type *Array of Structures* (AoS) vers une organisation *Structure of Arrays* (SoA). Dans la version de reference, chaque fourmi etait un objet 'ant' contenant simultanement :

- sa graine pseudo-aleatoire ;
- son etat charge / non charge ;
- sa position courante.

En Q2, ces informations sont stockees dans des tableaux separes :

— `m_x` ;
— `m_y` ;
— `m_is_loaded` ;
— `m_seed`.

L'interet attendu d'une telle transformation est classique en calcul haute performance :

— meilleure localite spatiale ;
— meilleure exploitation des lignes de cache ;
— acces memoire plus reguliers ;
— fondation plus favorable a une vectorisation automatique ulterieure.

4.2 Le paradoxe des performances : pourquoi SoA n'a pas apporte un gain magique

L'analyse du code et des premiers essais a montre un point important : la version AoS initiale etait deja relativement compacte et bien adaptee au cache. Les objets `ant` restaient de petite taille et etaient stockes de maniere contigue dans un `std::vector<ant>`. Le passage a SoA n'attaquait donc pas un probleme memoire catastrophique.

En parallele, le noyau `advance_one` reste difficile a vectoriser automatiquement car il contient :

— des branchements dependant de l'etat de la fourmi ;
— des choix exploration / exploitation ;
— un `do/while` dans la branche d'exploration ;
— des trajectoires qui dependent des donnees et des tirages pseudo-aleatoires.

Le rapport du compilateur, synthese dans `Q2/results/reviews/revue_vectorisation_ant.md`, confirme cette lecture. Aucun des signaux utiles n'est marque comme `optimized` ; au contraire, le compilateur remonte des causes telles que `couldn't vectorize loop` ou `number of iterations cannot be computed`. En pratique, SoA etait une condition necessaire pour assainir le stockage, mais certainement pas une condition suffisante pour obtenir une vectorisation SIMD complete.

4.3 Les vraies optimisations du code final

La baisse du temps d'execution observee en Q2 provient en realite de plusieurs optimisations concretes du point chaud `move_ants`.

4.3.1 Preallocation des structures auxiliaires

La methode `resize` prealloue les tampons de travail :

— `consumed_time` ;
— `active_loaded_ids` ;
— `active_unloaded_ids` ;
— `next_loaded_ids` ;
— `next_unloaded_ids`.

Ce choix evite des allocations dynamiques dans la boucle chaude et stabilise le cout de l'ordonnancement par rondes.

4.3.2 Separation des fourmis selon l'etat

Les fourmis actives sont traitees en deux groupes, `loaded` et `unloaded`. Cette separation supprime un branchement recurrent pour choisir le canal de pheromone et simplifie le flot de controle.

4.3.3 Reduction du cout du generateur pseudo-aleatoire

Le gain majeur vient de la specialisation des tirages aleatoires.

En Q1, le noyau chaud utilisait des helpers generiques :

```
rand_double(0., 1., seed)
rand_int32(1, 4, seed)
```

En Q2, ces appels sont remplaces par :

```
next_seed(seed)
rand_choice_01(seed)
rand_dir_14(seed)
```

Cette modification reduit le cout CPU pour plusieurs raisons :

— suppression de l'appel a `std::fmod` dans le noyau chaud ;
— specialisation exacte des deux usages reels du RNG ;
— reduction des calculs generiques lies aux bornes `min_val` / `max_val` ;
— meilleur potentiel d'integration directe par le compilateur.

Autrement dit, l'acceleration finale ne vient pas d'une vectorisation automatique complete, mais d'un allegement massif du calcul scalaire repete a tres haute frequence.

4.3.4 Autres ameliorations structurelles

Le code final retient egalement :

— le passage de `std::vector<bool>` a `std::vector<std::uint8_t>` pour l'etat des fourmis ;
— le passage de `std::size_t` a `std::uint32_t` pour les graines ;
— l'appel direct a `mark_pheronome_xy` pour simplifier legerement la mise a jour des pheromones.

4.4 Tableau comparatif Q1 vs Q2 dans le scope compute

La campagne finale Q2 a ete effectuee en mode headless, avec `T_render = 0`. Une comparaison directe des temps totaux Q1/Q2 serait donc trompeuse, puisque Q1 archive encore le cout du rendu. La comparaison pertinente porte sur le sous-total compute :

$$T_{\text{compute}} = T_{\text{move_ants}} + T_{\text{evap}} + T_{\text{pher_update}}.$$

Table 4 – Comparaison Q1 / Q2 sur le perimetre compute a partir des fichiers de `results/`.

Metrique	Q1 (ms)	Q2 (ms)	Gain relatif	Facteur d'acceleration
$T_{\text{move_ants}}$	15253.78	4611.60	-69.77 %	3.308x
T_{evap}	1664.29	1383.16	-16.89 %	1.203x
$T_{\text{pher_update}}$	38.92	30.62	-21.34 %	1.271x
T_{compute}	16956.99	6025.37	-64.47 %	2.814x

4.5 Reproductibilite et validation fonctionnelle

Les indicateurs de validation restent rigoureusement identiques entre Q1 et Q2 :

— `First_Iteration = 1236`

— `Food_KPI = 4353`

— `Checksum = 1479050492479337403`

Cette identite confirme que les optimisations introduites en Q2 ne modifient pas la logique sequentielle de reference. Le gain observe correspond bien a une acceleration d'execution, et non a une derivation du calcul.

4.6 Conclusion Q2

La Q2 montre une lecon importante en optimisation haute performance :

— le passage AoS -> SoA, pris isolenment, n'apporte pas necessairement un gain spectaculaire ;

— le rapport du compilateur est indispensable pour distinguer ce qui est vectorisable de ce qui ne l'est pas ;

— dans ce projet, les gains decisifs proviennent surtout de la simplification du noyau chaud et de la specialisation du generateur pseudo-aleatoire.

Le resultat est toutefois net sur le scope compute :

— le temps de deplacement des fourmis est reduit d'un facteur 3.308× ;

— le sous-total compute est reduit d'un facteur 2.814× ;

— la correction fonctionnelle est preservee.

Q2 constitue donc une etape techniquement reussie non pas parce qu'elle aurait force une vectorisation SIMD parfaite, mais parce qu'elle a permis de transformer un diagnostic de performance en optimisations concretes, mesurables et correctes.

5 Q3 : Parallelisation en Memoire Partagee (OpenMP)

5.1 Strategie generale et protocole experimental

La Q3 reprend la version SoA de Q2 et introduit une parallelisation en memoire partagee a l'aide d'OpenMP. Deux cibles principales sont identifiees :

— l'evaporation des pheromones, qui constitue une boucle reguliere et naturellement parallelisable ;

— le deplacement des fourmis, qui reste le noyau dominant mais pose des problemes d'ecritures concurrentes.

La campagne experimentale archivee dans `final_results/` suit le meme protocole que les etapes precedentes :

— warm-up prealable non comptabilise ;
— 5 repetitions mesurees par configuration ;
— rendu SDL desactive via `-gui 0` ;
— validation fonctionnelle par `Food_KPI` et `Checksum`.

Les valeurs archivees confirment une reproductibilite stricte entre Q2 et Q3 :

— `First_Iteration = 1236`
— `Food_KPI = 4353`
— `Checksum = 1479050492479337403`

L'absence de derive sur ces indicateurs montre que la parallelisation retenue ne modifie pas la logique sequentielle de reference et qu'aucune race condition observable n'est introduite dans la version finale.

5.2 Surmonter le goulet d'etranglement des verrous

Une premiere tentative naive consistait a proteger les structures partagees a l'aide de clauses `critical` et `atomic` directement dans le noyau de deplacement. Le schema etait conceptuellement du type :

```
#pragma omp parallel for
for (...) {
    ...
    #pragma omp critical
    phen.mark_pheronome_xy(...);

    #pragma omp atomic update
    ++cpteur_food;

    #pragma omp atomic capture
    index = nb_next_loaded++;
}
```

Cette approche garantissait bien la correction, mais elle s'est revelee defavorable en performance. La cause est structurelle : les fourmis se concentrent progressivement sur des chemins communs du paysage fractal. Les threads entrent alors en collision frequente sur les memes zones de pheromones, ce qui transforme la section critique en point de congestion global. Les operations atomiques sur les compteurs d'insertion des listes actives ajoutent une seconde source de contention.

Au lieu de calculer en parallele, les threads passent alors une part importante du temps a attendre l'acces aux ressources partagees. Ce comportement est typique d'un code irregulier a forte contention, et explique pourquoi une protection trop fine peut rendre une version OpenMP plus lente qu'une implementation sequentielle soigneusement optimisee.

La solution finale consiste donc a supprimer les verrous du chemin critique. Chaque thread accumule localement :

— les identifiants des fourmis qui restent actives pour le tour suivant ;
— les positions a marquer dans la carte de pheromones ;
— sa contribution locale au compteur de nourriture.

Une fusion sequentielle est ensuite realisee en fin de passe. Ce choix preserve la structure generale du scheduler par rondes de Q2, tout en deplacant la synchronisation hors de la boucle chaude.

5.3 Ordonnancement OpenMP retenu

La version finale du noyau de deplacement repose sur la structure suivante :

```
#pragma omp parallel if (nb_actives > 1024)
{
    #pragma omp for schedule(guided,64) nowait
    for (...) {
        ...
    }
}
```

Le choix de `parallel if (nb_actives > 1024)` est motive par le caractere iteratif du scheduler. En fin de ronde, le nombre de fourmis actives peut devenir faible. Dans ce cas, le cout de creation et de pilotage d'une equipe de threads peut depasser le travail utile. La garde conditionnelle permet donc de conserver une execution locale lorsque la granularite n'est plus suffisante.

Le choix de `schedule(guided,64)` repond a l'irregularite du deplacement :

— certaines fourmis convergent vite ;
— d'autres restent plus longtemps dans les boucles d'exploration ;
— la repartition spatiale est fortement non uniforme.

Un ordonnancement statique penaliserait alors le *load balancing*. L'ordonnancement guide attribue d'abord de gros blocs, puis reduit progressivement leur taille jusqu'a un minimum de 64 iterations. Ce compromis limite le cout du runtime tout en reequilibrant le travail en fin de boucle.

L'option `nowait` supprime enfin la barriere implicite en fin de `omp for`. Dans un code a charge variable, cela permet de reduire les attentes inutiles entre threads avant la fusion finale.

5.4 Resultats et analyse quantitative

Le temps sequentiel de reference est pris dans `final_results/Q2_timings_breakdown.csv` :

$$T_s = 6026.461186 \text{ ms}$$

Le speedup et l'efficacite sont definis par :

$$S(p) = \frac{T_s}{T_p} \qquad E(p) = \frac{S(p)}{p}$$

Table 5 – Synthese des performances Q3 a partir de `final_results/Q3_openmp_scaling.csv`.

Threads p	Temps moyen T_p (ms)	Speedup $S(p)$	Efficacite $E(p)$
1	7349.997	0.820	0.820
2	5664.447	1.064	0.532
4	4878.286	1.235	0.309
8	7591.974	0.794	0.099

Le premier constat est l'existence d'un surcout a 1 thread : la version OpenMP est plus lente que Q2. Ce resultat est attendu et provient du runtime OpenMP lui-meme, ainsi que de la creation et de la reinitialisation des buffers locaux a chaque appel du noyau de deplacement. Cette surcharge est le prix a payer pour rendre le code parallelisable.

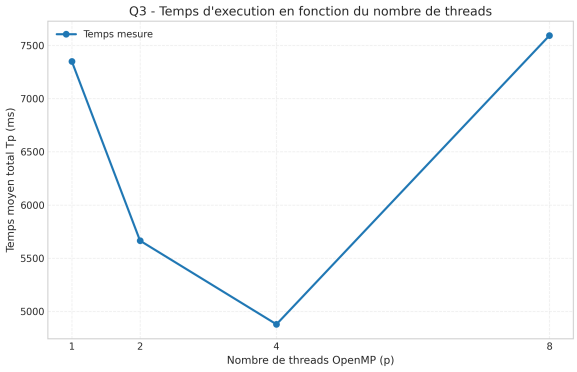
En revanche, le passage a 2 puis 4 threads devient benefique. La meilleure configuration de la campagne est atteinte avec 4 threads :

— $T_4 = 4878.286$ ms ;
— $S(4) = 1.235$ par rapport a la reference sequentielle Q2 ;
— gain global d'environ 19% par rapport a Q2.

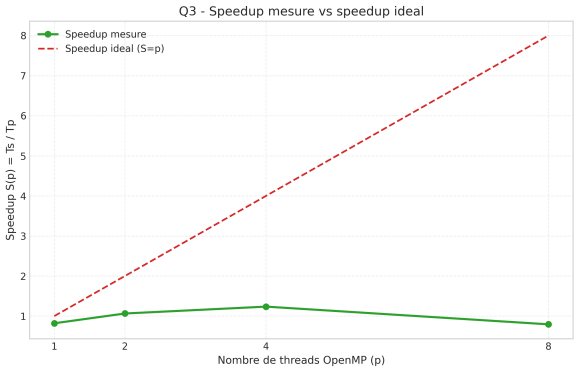
Ce comportement est coherent avec la machine de test, qui dispose de 4 coeurs physiques. A cette echelle, l'evaporation et le deplacement des fourmis beneficient d'une mise en parallel efficace sans saturer totalement les ressources physiques.

5.5 Analyse des figures de scalabilite

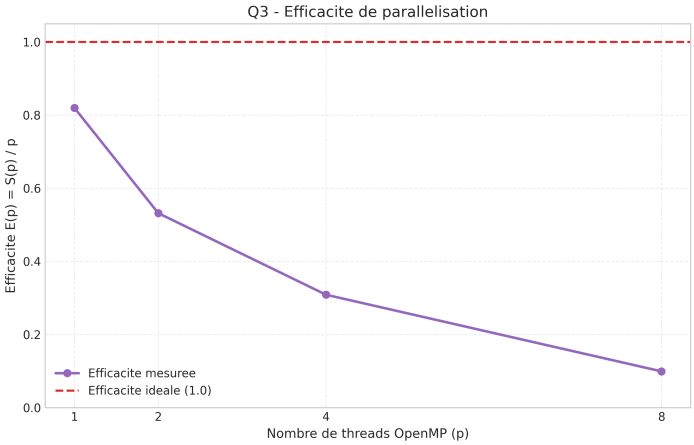
Les graphiques generes a partir des fichiers de `final_results/` sont inseres ci-dessous. Ils illustrent respectivement l'evolution du temps d'execution, du speedup et de l'efficacite.



Temps d'execution en fonction du nombre de threads.



Speedup mesure compare au speedup ideal.



Efficacite de parallelisation.

FIGURE 1 – Graphiques de scalabilite Q3 generes a partir des resultats finaux.

L'analyse scientifique de ces courbes fait apparaître trois regimes distincts.

5.5.1 Regime 1 thread : overhead pur

Le point $p = 1$ reste en dessous de la reference sequentielle Q2, avec $S(1) = 0.820$. Ce comportement traduit le cout fixe du runtime OpenMP, la gestion des buffers locaux et la logique de fusion introduite pour supprimer les verrous dans le noyau chaud. La parallelisation n'est donc rentable que lorsque la quantite de travail disponible suffit a amortir cette infrastructure supplementaire.

5.5.2 Regime utile : 2 a 4 threads

Entre 2 et 4 threads, le temps decroit effectivement et le speedup croît. On observe cependant un eloignement progressif par rapport a la droite ideale $S(p) = p$. Cet ecart s'explique par plusieurs facteurs :

— une partie du code reste sequentielle, notamment la fusion finale des buffers locaux ;

— l'ordonnancement OpenMP et la gestion des equipes de threads introduisent un overhead non nul ;

— la charge reste irreguliere, meme avec `guided`, ce qui empeche un equilibre parfait.

Autrement dit, la Q3 valide un gain de performance reel, mais elle ne peut pas atteindre la courbe ideale en raison des contraintes structurelles du probleme.

5.5.3 Regime 8 threads : plafond materiel et Hyperthreading

La degradation a 8 threads est physiquement cohérente avec la machine de test. Celle-ci possede 4 coeurs physiques et 8 coeurs logiques. Le passage de 4 a 8 threads n'ajoute donc pas de nouvelles unites de calcul physiques ; il active essentiellement le SMT / Hyperthreading.

Dans une charge deja intensive en calcul et en trafic memoire, les coeurs physiques saturent rapidement :

— les ALU et unites de calcul sont deja sollicitees ;
— les caches L1/L2 sont fortement utilises ;
— la bande passante memoire devient plus disputee.

Les threads logiques supplementaires n'apportent alors pas un debit additionnel significatif. Ils introduisent au contraire davantage de concurrence sur les memes ressources physiques, ce qui explique :

— la chute de l'efficacite a $E(8) \approx 0.099$;
— l'augmentation de la variabilite experimentale ;
— la perte de performance globale par rapport a 4 threads.

5.6 Conclusion Q3

La Q3 montre qu'une parallelisation OpenMP reussie ne repose pas seulement sur l'ajout de pragmas, mais sur une maitrise de la contention et de la granularite du travail.

Les conclusions principales sont les suivantes :

— l'approche naive par `critical` et `atomic` dans le noyau chaud est defavorable ;
— la strategie a buffers locaux elimine le goulet d'etranglement des verrous tout en conservant la structure generale du code Q2 ;
— l'ordonnancement `guided` et la garde `if (nb_actives > 1024)` permettent de contenir l'overhead du runtime ;
— le meilleur compromis experimental est obtenu a 4 threads, correspondant au nombre de coeurs physiques ;
— 8 threads mettent en evidence la limite de l'Hyperthreading pour ce type de charge compute-bound et memory-bound.

En synthese, la Q3 valide un gain mesurable par parallelisation en memoire partagee, avec une acceleration maximale d'environ 1.235× par rapport a Q2. Le resultat reste loin du speedup ideal, mais il est scientifiquement coherent avec la loi d'Amdahl, avec l'irregularite du noyau de deplacement, et avec les limites materielles de la plateforme.